Lezione 6 - API

Introduzione

Nella lezione precedente abbiamo analizzato il formato JSON e abbiamo visto perché rappresenti lo standard ideale per lo scambio dei dati tra client e server: è leggero, facilmente leggibile e perfettamente integrabile nelle architetture web moderne.
A partire da questo punto, possiamo approfondire come tali dati vengano effettivamente trasmessi e gestiti tramite le **API**, che costituiscono il meccanismo attraverso cui applicazioni diverse comunicano tra loro in modo strutturato e controllato.

API (Application Programming Interface)

Un'API, ovvero un'**Interfaccia di Programmazione di un'Applicazione**, è:

- **un meccanismo che definisce un insieme di regole precise affinché due componenti software possano comunicare e collaborare tra loro.**

In altre parole, un'API funziona come un **contratto**:

- **stabilisce quali operazioni possono essere richieste, come devono essere formulate e quali risposte ci si deve aspettare.**
- Grazie a questo contratto, **due sistemi possono interagire senza conoscere i dettagli interni del loro funzionamento.**

L'idea alla base delle API si manifesta soprattutto in un contesto oggi fondamentale:

- **la comunicazione tra applicazioni distinte, che spesso risiedono su computer diversi e dialogano attraverso una rete come Internet.**

L'API come Ponte tra Programmi Diversi

La metafora più intuitiva per comprendere il ruolo di un'API è quella del **ristorante**:

- **Il Cliente (tu / il client):**
  - **rappresenta l'applicazione che ha bisogno di un'informazione o di un servizio.**

- **La Cucina (il server):**
  - **è il sistema che contiene dati e funzionalità, ma che non è direttamente accessibile.**
- **Il Cameriere (l'API):**
  - **è l'intermediario che riceve la tua richiesta, la traduce nelle modalità corrette e la consegna alla cucina.**
  - Poi ti riporta la risposta (il piatto).

Come nel ristorante tu non entri mai in cucina, allo stesso modo un'applicazione non accede direttamente alle funzioni interne di un server.
Si affida all'API, che garantisce una comunicazione ordinata e sicura.

Grazie a questo ruolo intermedio:

- **il client formula richieste seguendo le regole dell'API (il "menu");**
- **il server elabora la richiesta;**
- **l'API restituisce una risposta chiara, spesso in formato JSON.**

Esempi di API nella pratica

Le API sono ovunque nello sviluppo moderno. Alcuni esempi comuni:

- **Login con Google o Facebook:**
  - Il sito che stai usando non vede mai la tua password. È l'API del provider che verifica l'identità e comunica solo il risultato.
- **Applicazioni Meteo:**
  - L'app non misura direttamente il tempo atmosferico.
  - Interroga un server remoto tramite API e riceve i dati aggiornati.

In tutti questi casi, l'API consente a sistemi diversi di cooperare in modo standardizzato, sicuro e prevedibile, nascondendo la complessità del lato server e rendendo semplice l'integrazione tra componenti software.

L'API come "Manuale d'Istruzioni" di un Software

Il concetto di API non riguarda solamente la comunicazione via rete.
In senso più ampio, un'API definisce

- **come uno sviluppatore può accedere e utilizzare le funzionalità messe a disposizione da una libreria, da un framework o dal sistema operativo**

all'interno del proprio programma.

Possiamo immaginare l'API come una vera e propria **cassetta degli attrezzi**:

- non solo contiene gli strumenti che il software offre, ma include anche il **manuale** che spiega come usarli correttamente.
- **Grazie all'API, lo sviluppatore può integrare funzioni complesse senza dover conoscere nei dettagli la loro implementazione interna.**

Esempio pratico: la libreria React

React fornisce due API distinte per creare componenti:

1. **Componenti a Classe (Class Components)**
   È l'API "storica" di React. Lo sviluppatore definisce una classe JavaScript che estende `React.Component` e utilizza metodi specifici, come:
   - `render()` → **per descrivere cosa mostrare a schermo;**
   - `componentDidMount()` **e altri metodi del ciclo di vita → per gestire eventi particolari;**
   - `this.state` e `this.setState()` → **per manipolare lo stato interno del componente.**
2. **Componenti Funzionali con Hooks (Functional Components)**
   - **È l'API moderna**.
   - I componenti sono funzioni semplici che, tramite gli **Hooks**, possono gestire lo stato o altre capacità avanzate:
     - `useState()` per memorizzare valori interni,
     - `useEffect()` per gestire effetti collaterali come fetch o timer.

Le due modalità consentono di ottenere lo stesso risultato (creare componenti), ma sono **API diverse**:

- **due modi differenti di comunicare con le funzionalità interne di React, senza dover conoscere la complessità del suo funzionamento interno.**

Un altro esempio quotidiano è rappresentato dall'**API del DOM**: i browser espongono funzioni come `document.getElementById()` per manipolare elementi di una pagina web. Anche qui l'API funge da livello di astrazione che permette agli sviluppatori di lavorare senza conoscere in dettaglio il motore del browser.

Il ruolo dell'API come livello di astrazione

Che si parli di un “cameriere” che trasporta le richieste verso un server remoto o di un “manuale d’uso” per una libreria, il ruolo dell’API rimane lo stesso:

- **fornire un livello di astrazione che semplifica l’interazione con sistemi complessi.**

L’API **nasconde la complessità interna** del sistema.
Non è necessario conoscere la struttura dei server di Google per usare Google Maps, così come non occorre studiare il codice sorgente di Vue o React per costruire un’interfaccia.
È sufficiente seguire le regole definite dall’API, e questa garantirà un risultato preciso.

In sintesi, un’API rappresenta una promessa chiara nel mondo del software:

> **“Se rispetti le regole del mio contratto, ti fornirò un risultato affidabile, senza che tu debba sapere come lo ottengo.”**

Il principio che accomuna tutte le API: l’Astrazione

Le API non sono solo un meccanismo tecnico, ma un approccio strategico alla progettazione del software. Offrono infatti tre vantaggi fondamentali:

● Riutilizzo (Reusability)

Una volta che una funzionalità è stata sviluppata e resa disponibile tramite un’API (esempio: un sistema di pagamento, un motore di ricerca o un servizio di geolocalizzazione), può essere riutilizzata da altre applicazioni infinite volte.
Questo evita di “reinventare la ruota” e riduce drasticamente tempi e costi di sviluppo.

● Modularità (Modularity)

Le API favoriscono sistemi composti da moduli indipendenti.
Ogni componente svolge un compito specifico e comunica con gli altri tramite API chiaramente definite.
Questo approccio facilita aggiornamenti, manutenzione e scalabilità, poiché è possibile modificare un singolo modulo senza compromettere l’intero sistema.

● Sviluppo Parallelo (Parallel Development)

Poiché l'API funge da contratto, i diversi team possono lavorare in contemporanea:

- **il team** frontend **può sviluppare interfacce e logiche di visualizzazione;**
- **il team** backend **può definire la logica e i dati del server.**

È sufficiente che entrambi conoscano il "contratto" dell'API per procedere in parallelo. L'intero sistema può avanzare più velocemente e in modo più coordinato.

Classificazione delle API

Le API possono essere raggruppate in base a vari criteri:

- la tecnologia utilizzata,
- il modo in cui comunicano o il livello di accessibilità.
  Comprendere queste distinzioni è essenziale per orientarsi nello sviluppo moderno, dove le API rappresentano il principale punto di contatto tra servizi, applicazioni e componenti software.

1. API Web(HTTP - based)

La categoria più diffusa oggi è costituita dalle **API Web:**
ovvero quelle che utilizzano il protocollo **HTTP**, lo stesso usato dai browser per visitare un sito Internet.
Questo tipo di API sfrutta gli standard del Web per permettere a sistemi anche molto diversi tra loro di comunicare in modo semplice e interoperabile.

All'interno di questa famiglia troviamo approcci architetturali differenti, ognuno con caratteristiche e scopi specifici.

SOAP (Simple Object Access Protocol)

È uno dei primi standard per creare API.
Si basa su un protocollo molto rigido, con regole severe sulla struttura dei messaggi.
Caratteristiche principali:

- **utilizza principalmente XML;**
- **offre robustezza e funzionalità avanzate, soprattutto in ambito entreprise;**

- **è più complesso e produce messaggi più “pesanti”.**

• REST (REpresentational State Transfer)

Non è uno standard, ma uno stile architettonico.

- È più flessibile e intuitivo rispetto a SOAP, e sfrutta i metodi nativi di HTTP (GET, POST, PUT, DELETE, …).
- **Di solito utilizza il formato JSON, più leggero e facile da trattare rispetto a XML.**
- È attualmente lo stile dominante per le API moderne, soprattutto nel web e nel mobile.

• GraphQL

Sviluppato da Facebook, non è un protocollo ma un linguaggio di query per API.
La sua particolarità è che permette al client di chiedere **esattamente** i dati necessari e nella forma desiderata.
A differenza di REST:

- **espone in genere un solo endpoint;**
- **elimina l’over-fetching (ricevere più dati del necessario);**
- **evita l’under-fetching (dover fare più chiamate per ottenere tutto).**

È molto utile in applicazioni complesse o nei sistemi con un gran numero di relazioni tra entità.

API Sincrone vs. Asincrone

Questa distinzione riguarda **come** il client riceve la risposta dal server.

API Sincrone

Seguono il modello classico *richiesta–risposta*.
Il client invia la richiesta e rimane in attesa finché il server non ha completato l’operazione e fornito l’esito finale.

- È il comportamento tipico delle API REST.
- **L’applicazione resta “in pausa” finché non arriva la risposta.**

È ideale per operazioni rapide.

API Asincrone

- **Il client invia la richiesta e non resta bloccato.**
- **Il server risponde subito con un messaggio che conferma la presa in carico ("ok, sto lavorando"), ed elabora il resto in background. Quando l'operazione è terminata, il server invia il risultato tramite meccanismi come:**
- **callback**,
- **webhook**,
- **eventi**.

Questo modello è perfetto per operazioni lunghe, come:

- **elaborazione video,**
- **generazione di report,**
- **analisi dati complesse.**

API Pubbliche vs Private

Questa classificazione riguarda chi può usare l'API.

API Pubbliche (o Esterne)

Sono aperte a sviluppatori esterni e pensate per essere integrate in applicazioni di terze parti.
Esempi tipici:

- API di Google Maps,
- API di Twitter o Instagram,
- API meteo.

L'obiettivo è favorire l'integrazione di servizi e funzionalità in applicazioni diverse, espandendo l'ecosistema dell'azienda.

API Private (o Interne)

Sono utilizzate esclusivamente all'interno di una singola organizzazione. Servono a far comunicare servizi interni, microservizi o sistemi aziendali eterogenei.
Non sono esposte al pubblico, ma seguono comunque gli stessi principi delle API pubbliche:
documentazione chiara, contratti ben definiti e comunicazione standardizzata.

Sono fondamentali in architetture moderne basate su:

- **microservizi,**
- **orchestrazione interna,**
- **scalabilità modulare.**

Endpoint e payload

Indipendentemente dalla tecnologia adottata o dal modello architetturale, tutte le API web condividono alcuni concetti fondamentali che regolano il modo in cui client e server comunicano.
Tra questi, due elementi sono particolarmente importanti:

1. **gli endpoint**
2. **il payload**.

Endpoint

Un *endpoint* rappresenta il punto di accesso a una determinata risorsa esposta da un'API.
In termini pratici:

- **è un URL che identifica in maniera univoca "dove" il client deve inviare la richiesta.**
  Possiamo considerarlo come l'indirizzo di una casa:
  indica con precisione al client il luogo in cui trovare l'informazione desiderata o dove inviare dei dati da elaborare.

Alcuni esempi:

- `https://api.example.com/users`
  Endpoint che restituisce la collezione completa degli utenti.
- `https://api.example.com/products/123`
  Endpoint che fa riferimento al prodotto con identificativo `123`.

Ogni endpoint è progettato per svolgere una funzione specifica, come elencare risorse, recuperarne una singola, crearne di nuove o modificarne di esistenti.
La chiarezza con cui vengono definiti è essenziale affinché il client sappia esattamente come interagire con l'API.

Payload

Il *payload* è:

- **il contenuto informativo vero e proprio che viene scambiato tra client e server.**
  Se l'endpoint rappresenta l'indirizzo, il payload è il messaggio contenuto nella "busta":
- **i dati che devono essere elaborati o restituiti.**

Request Payload

- **È il contenuto inviato dal client al server, generalmente all'interno del body di una richiesta POST, PUT o PATCH.**
  Ad esempio, nella creazione di un nuovo utente, il request payload potrebbe includere:

JSON

```json
{
  "nome": "Mario",
  "email": "mario@example.com"
}
```

Response Payload

- **È il contenuto inviato dal server al client come risposta.**
  Nel caso di una richiesta **GET** all'endpoint `/users`, il response payload includerà la lista degli utenti disponibili.

Formati più comuni del payload

I dati contenuti nel payload possono essere rappresentati in vari formati, tra cui:

- **JSON (JavaScript Object Notation)**
  È oggi il formato dominante nelle API REST per leggerezza, semplicità e facilità di utilizzo sia lato client che server.
- **XML (eXtensible Markup Language)**
  Storicamente associato alle API SOAP, è più verboso ma altamente strutturato e rigoroso.
- **Form-data**
  Formato utilizzato soprattutto dai browser per l'invio dei dati dei moduli HTML, inclusi file e immagini.